

Academic year
2025 - 2026

Exam Programming: PINN

Using Physics Informed Neural Networks to solve nonlinear Partial Differential Equations

Des De Borger



University of Antwerp
| Faculty of Science

Contents

1. Introduction	1
2. Damped Harmonic Oscillator	3
2.1. Analytic solution	3
2.2. Understanding an the PINN example	3
2.2.1. Questions on the method	3
2.2.2. Questions on the code	8
2.3. Improving the example code	12
2.3.1. Code improvements	12
2.3.2. Code demonstration	12
2.3.3. Hyperparameter optimisation	12
2.4. Inverse problem	12
2.4.1. Underdamped case	12
2.4.2. Critically damped case	12
2.4.3. Overdamped case	12
2.4.4. Statistical analysis	12
2.4.5. Discussion	12
3. Burger's Equation	13
3.1. Ground truth solving strategies	13
3.1.1. Numerical PDE solvers	13
3.1.2. Cole-Hopf Transformation	13
3.2. PINNs for the Burger's equation	13
3.2.1. Code implementation	13
3.2.2. Code demonstration	13
3.2.3. Hyperparameter optimisation	13
3.3. Inverse problem	13
3.3.1. Underdamped case	13
3.3.2. Critically damped case	13
3.3.3. Overdamped case	13
3.3.4. Statistical analysis	13
3.3.5. Discussion	13
4. Conclusion	15
A. Equation derivations	17
B. Figures	19

1. Introduction

2. Damped Harmonic Oscillator

2.1. Analytic solution

$$my''(t) + cy'(t) + ky(t) = 0 \quad (2.1)$$

2.2. Understanding an the PINN example

2.2.1. Questions on the method

Question 1

Identify the total loss function in the code and write its expression down in your report. Identify the different contributions to the loss function.

The total loss function has three terms and is displayed below:

$$\mathcal{L}_{tot} = \mathcal{L}_{data} + \lambda_{physics} \mathcal{L}_{physics} + \lambda_{i.c.} \mathcal{L}_{i.c.} \quad (2.2)$$

The first term is the mean squared error of the predicted data compared to the 15 noisy observation points:

$$\mathcal{L}_{data} = \frac{1}{M} \sum_{j=1}^M (\hat{y}(t_j) - y_j)^2 \quad (2.3)$$

with $j = 1, 2, \dots, 15$. This term is the total loss function of a standard, non-PI NN and gives the model direct information about the correct solution of the equation with the relevant boundary conditions, but doesn't check for any non-physical behavior.

The second term is the mean squared residual of the ODE (see equation 2.1) at the 200 evenly spaced collocation points:

$$\mathcal{L}_{physics} = \frac{1}{N} \sum_{i=1}^N (m\hat{y}''(t_i) + c\hat{y}'(t_i) + k\hat{y}(t_i))^2 \quad (2.4)$$

with $i = 1, 2, \dots, 200$. This term gives the model information about how much the predicted solution violates the mechanics of the system, without checking if the predicted solution matches the observed data.

The third term is the sum of squared errors of the two initial conditions:

$$\mathcal{L}_{i.c.} = (\hat{y}(0) - y_0)^2 + (\hat{y}'(0) - dy_0)^2 \quad (2.5)$$

This term enforces the initial conditions and applies only to the very first point $t = 0$.

The constants $\lambda_{physics}$ and $\lambda_{i.c.}$ determine the relative weight of every term in the loss function. Because it is important that the initial conditions are met, $\lambda_{i.c.}$ is chosen at a much larger value than $\lambda_{physics}$. This is why in the given program, the values are set at $\lambda_{physics} = 0.01$ and $\lambda_{i.c.} = 10$.

Question 2

Is the inclusion of the initial condition loss essential for this particular example? When would it be essential?

To investigate the effect of the initial condition term, the term can be dropped out of the loss function to compare the results. Figure 2.1 shows the predicted solutions of the ODE if the $\mathcal{L}_{i.c.}$ is included and excluded. In this specific case, there does not seem to be a big difference which indicated that the initial condition loss is not essential. This is because the noisy observations point, which also contribute to the loss function, are close enough to $t = 0$ to give enough information for the model to accurately predict the initial conditions.

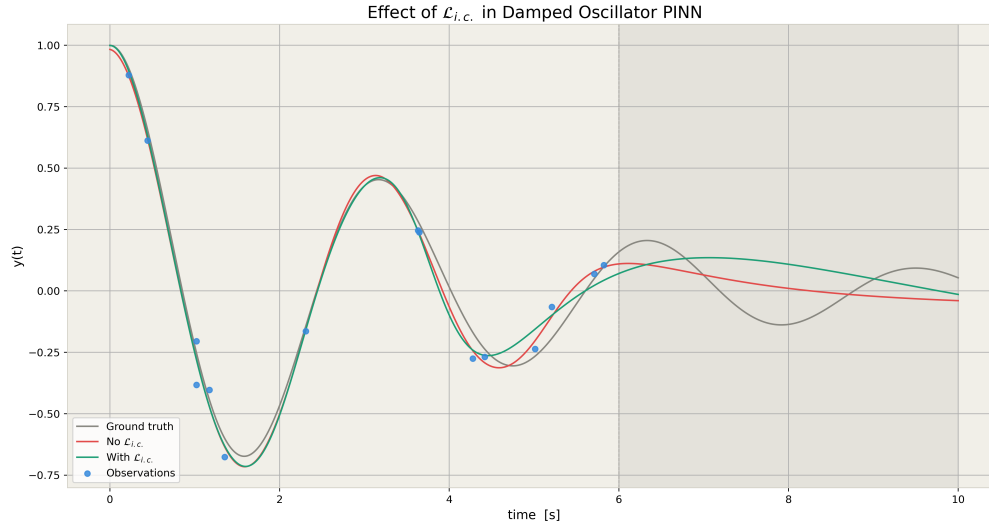


Figure 2.1.: Predicted solutions to the ODE of the $\mathcal{L}_{i.c.}$ is included and excluded. In this specific case, the $\mathcal{L}_{i.c.}$ is not absolutely necessary, because the noisy observation points give enough information for the model to accurately predict the initial conditions.

We do not always have access to observations close to the initial time t_0 . However, if the initial conditions are known and need to be enforced, $\mathcal{L}_{i.c.}$ is a crucial term in the total loss function.

Question 3

The loss function used here is based on the mean squared errors (MSE) between predictions and data points or initial condition, and on the mean squared residual (MSR) of the PDE for the collocation points. Why are these specific functional forms used, and would there be alternative functions that could replace the MSE or MSR?

The mean square error and mean square residuals are the most commonly used loss functions for regression models. Any loss function has to be symmetrical and positive for all possible values, so that errors cannot cancel each other out. Furthermore, it is desired that the loss function is differentiable so that its gradient can be computed.

The MSE and MSR check both of these boxes. However, since the errors are squared before averaging, these functions heavily punish and are sensitive to large outliers. This can lead to problems when very noisy data points are included in the calculation for the loss function. However, this effect is good for the residuals since it is essential that the predicted solution does not break the laws of physics. There are alternative loss functions with different (dis)advantages [1]:

Mean Absolute Error The Mean Absolute Error (MAE) does not average the squares, but the absolute distances:

$$\mathcal{L}_{MAE} = \frac{1}{M} \sum_{j=1}^M |\hat{y}(t_j) - y_j| \quad (2.6)$$

Because the errors are not squared, this loss function is less sensitive to large outliers, which can make it more robust. However, it is not differentiable at zero, which can lead to problems for some optimisation algorithms.

Huber Loss The Huber Loss is a hybrid of the MAE and the MSE which is differentiable, but less sensitive to outliers than the MSE. It is defined as follows:

$$L_{\delta}(x) = \begin{cases} \frac{1}{2}x^2 & \text{if } |x| \leq \delta \\ \delta (|x| - \frac{1}{2}\delta) & \text{if } |x| \geq \delta \end{cases} \quad (2.7)$$

The Huber Loss behaves like the MSE for $|x| < \delta$ and like the MAE for $|x| > \delta$. The hyperparameter δ can be tuned to make the loss function more or less sensitive to outliers.

Question 4

What is the type of neural network used in this example? How many input nodes, output nodes and hidden layers does it have? How many parameters are then being optimized? What type of activation function is used and why? What quantity or quantities are provided at the input, and what quantity or quantities do we extract from the output of the network? Are the dimensions of this Neural Network architecture adequate, too small or too large for this example? How would you determine this?

We are trying to solve an ODE, which means to solve for a function $y(t)$. This function takes one input and returns one output, which means the network will also need to have one input and one output, representing the time and the function value respectively.

A dense network is used in this specific case. The parameters for the hidden layers can be given as inputs, in the example script there are 4 hidden layers of 32 nodes. For every connection between nodes has a weight, and every node (except the input) has a bias. From this we can calculate the total parameter count:

$$\begin{aligned} n &= \text{First layer weights} + \text{First layer bias} \\ &\quad + \text{Hidden layer weights} + \text{Hidden layer bias} \\ &\quad + \text{Last layer weights} + \text{Last layer bias} \\ &= (1 \cdot 32 + 32) + 3(32 \cdot 32 + 32) + (1 \cdot 32 + 1) = 3265 \text{ parameters} \end{aligned} \quad (2.8)$$

The dimensions of the network (the depth and layer size) are hyperparameters which should be tuned according to the specific problem needs to be solved by the model. Defining the optimal value for hyperparameters such as the dimensions is a relatively subjective and empirical process, since there are no general formulas that give the optimal hyperparameter set for every problem.

The activation function used is a tanh function. This choice is important; we will be calculating the first and second derivatives of the output y with respect to the input t , which means the activation function needs to be differentiable at least twice. for this reason, ex. ReLu is not suitable for this network.

Judging from the predicted solution, the model is large enough to make an accurate prediction for the solution to the equation. To optimise the network dimensions further, one can choose to implement an optimising algorithm which tunes the hyperparameters based on a validation set. This way, the accuracy and training time can be optimised in a reproducible way [5]. Examples possible methods include:

- Random search: for every hyperparameter, a range of evenly spaced values is defined. A model is trained for every single possible combination and their performances are compared.
- Random search: hyperparameter sets are chosen at random with user-defined ranges, and the best performing model is chosen from the resulting set of models.
- Bayesian optimization: a probabilistic sequential algorithm that optimises an unknown, computationally heavy function (such as the model performance).

Question 5

How many input samples are provided to the neural network model during each of its training cycles or epochs? What kind of samples are provided? Does the network receive again for each epoch the same exact samples? Would there be a better way to provide samples to this network for training over the various epochs?

The model receives the same 15 noisy observation points every single epoch. These points are used to calculate $\mathcal{L}_{data}$. Furthermore, the model receives the initial conditions to calculate $\mathcal{L}_{i.c.}$. The $\mathcal{L}_{physics}$ is calculated only using the residual of equation 2.1, for which no inputs are needed.

This is a suboptimal approach, since the model could simply memorize the 15 given points. A better approach would be to randomize the 15 points every epoch to avoid overtraining on a specific dataset. Furthermore, it would also be good practise to stop the inputting of datapoints after a defined number of epochs, so that the model is solely trained based on the equation residuals. This is even an even safer approach to avoid overtraining and is known as *curriculum training*. [6]

Question 6

The example uses the Adam algorithm as the so-called optimizer. What is the purpose of this optimizer? What is specific about the Adam algorithm, and what are its main hyper-parameters? Are there viable alternatives for the Adam optimizer?

An optimizer is a tool to accelerate the gradient descent by taking previously calculated gradients into account [2]. This way, the optimizer can avoid oscillations and avoid local minima. Adam is actually a combination of two alternative optimisers:

1. Momentum-based optimising [3]: this technishe incorporates the gradients of previous epochs by calculating an exponentially weighted moving average. If the gradient at epoch t is given by w_t , the momentum-based gradient m_t is calculated iteratively as follows:

$$m_t = \beta m_{t-1} + (1 - \beta) \nabla_{w_t} \mathcal{L}_{tot} \quad (2.9)$$

where β is the momentum factor, typically chosen as 0.9. This parameter determines how much the previous gradients need to be taken into account. The weights are then updated:

$$w_t = w_{t-1} - \alpha m_t \quad (2.10)$$

2. Root Mean Square Propagation [4]: this technique uses an exponentially weighted moving average of squared gradients to optimise the learning rate α . The learning rate is divided by a factor $\sqrt{v_t}$, the root (weighted) mean square of the gradients. This is also calculated iteratively:

$$v_t = \gamma v_{t-1} + (1 - \gamma) (\nabla_{w_t} \mathcal{L}_{tot})^2 \quad (2.11)$$

The weights are then updated:

$$w_t = w_{t-1} - \frac{\alpha}{\sqrt{v_t} + \epsilon} \nabla_{w_t} \mathcal{L}_{tot} \quad (2.12)$$

A small value is added in the denominator to avoid division by zero.

Both m and v are biased towards zero since they are initialized at zero. They need to be corrected:

$$\begin{aligned}\hat{m}_t &= \frac{m_t}{1 - \beta^t} \\ \hat{v}_t &= \frac{v_t}{1 - \gamma^t}\end{aligned}\tag{2.13}$$

The Adam optimiser combines these two techniques and updates the weights as follows:

$$w_t = w_{t-1} - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t\tag{2.14}$$

The Adam optimiser is widely used since it generally has an efficient performance with relative little hyperparameters; only α , β , and γ need to be defined. Both the learning rate and the gradient itself are made adaptive which helps to find the global minimum of the loss function more effectively. Alternative optimisers include the momentum-based and RMSprop techniques as discussed above, or Adamax (a variant of Adam based on the infinity norm) and Adagrad (which accumulated the gradients instead of using the weighted average).

Question 7

The example also uses an additional optimizer: a learning rate scheduler. What does this scheduler do? Is it a necessary ingredient for training a neural network? What are its main hyperparameters? Are there alternatives that can be usefully applied in this example?

The learning rate scheduler StepLR reduces the the learning rate with a factor γ after every n epochs, which are tunable hyperparameters. This adds a global decay, which can optimize the convergence in the different stages of optimisation. When the number of epochs is still small and the parameter set is still far from the optimum, the learning rate is big, but as the number of epochs gets bigger and the network converges towards the best set of parameters, the learning rate has to decrease as well in order to avoid oscillations around the minimum. This approach lead to a further acceleration of the gradient descent.

There exist many different alternative learning rate schedulers, such as:

1. ExponentialLR: similar to StepLR but applies a small decay factor γ after every epoch, leading to an exponentially decreasing learning rate.
2. CosineAnnealingLR: the learning rate follows a cosine curve which converges to a minimum value.
3. ReduceLROnPlateau: adaptive scheduler which applies a decay factor γ when improvement stagnates.
4. CyclicalLR: non-traditional learning rate scheduler which periodically increases and decreases the learning rate in order to avoid local minima.

In this specific case, StepLR or ExponentialLR are good choices to effectively train the model.

Question 8

How many epochs is the network trained for? How would you determine whether you have trained for too little or too many epochs?

The model is trained for 8000 epochs. In order to determine if this is too many/too little, we need to compare the test and training loss function curves as a function of the number of epochs. When these

two curves start to diverge, it means the model is overfitting on the training set. The model is then learning meaningless details in the test set. In the provided script no validation dataset is implemented, which makes it impossible to detect overfitting in this way. A proper implementation would reserve a separate set of points, unseen during training, against which the loss is evaluated each epoch.

Question 9

Would it make sense to have a set of validation points? How would these look like? How would you use them?

This script does not have a set of validation points, which makes it difficult to gauge the accuracy of the model while taking the possibility of overtraining into account. The most representative way to determine the accuracy of the solution is to compare it directly to the analytic, exact solution. This can be done by defining a set of many evenly spaced values for t and calculating the analytic solution $y(t)$ at these points. This acts as the ground truth of the model. Then, the MSR between the predicted and ground truth datapoints can be calculated to give an idea of the accuracy of the model.

It could also be wise to split the validation points into two sets in order to more specifically investigate the model's ability to extrapolate beyond the range in which observation points were supplied. The set would then be split up into a set of extrapolated and non-extrapolated points. Then, the mean squared errors of both sets can be compared to analyse how much worse the predictions for the extrapolated region are.

2.2.2. Questions on the code

Question 1

Why are these two lines of code good, or bad, for:

```
torch.manual_seed(42)
np.random.seed(42)
```

These lines of code set the seed for the random number generation in the script. Using the same seed every time makes it so that the noisy observation points are exactly the same when the script is ran. This can make it easier to debug the script when a specific combination of observation points causes a bug.

It is however also important to occasionally test the script with different seeds. The model architecture and the script could become optimised for this specific seed if other seeds are never tested.

Question 2

What is the purpose of the `np.ndarray` statements in this function definition:

```
def analytic(t: np.ndarray) -> np.ndarray:
```

This is Python's type annotation syntax, as prescribed by PEP484 [9]. It tells the user which datatypes the input and output need to be. No typechecking happens at runtime, so this statement doesn't explicitly raise a `TypeError` if an object of the wrong type is passed. It serves as documentation and can help the developer to keep track of the precise definitions of every variable.

Question 3

The data points are created randomly and then sorted before being offered to the neural net, using this statement:

```
t_obs = np.sort(np.random.uniform(0.1, T_TRAIN, N_OBS))
```

Is this good or bad?

Randomizing the datapoints is good practise. A periodic set of points could lead the model to memorize specific patterns, instead of predicting the general solution to the initial condition. However, as said before, this script always passes the same set of random points which negates the advantage of randomizing the points in the first place.

The sorting of the points has no effect, since we are taking the MSE between the observation points and predicted points. The order in which the errors are summed has no effect on the loss function.

Question 4

The collocation points are created uniformly on a fixed grid, using this statement:

```
t_col = np.linspace(0.0, T_EXTRAP, N_COL)
```

Is this good or bad?

Using a uniform grid for the collocation points is not optimal, the model can fine-tune to the specific spacing of the collocation grid which is not desirable. It is better practise to generate the collocation points randomly at every epoch. By constantly regenerating a set of collocation points, any bias towards one region is also avoided.

Questions 5 and 6

What is this statement doing? Is it necessary?

```
optimiser.zero_grad()
```

What are these statements doing in the training cycle? Is their order important?

```
loss_total.backward()
optimiser.step()
scheduler.step()
```

This sequence of commands updates the model weights by calculating the gradients through back-propagation. `loss_total` is a 0-dimensional PyTorch tensor representing the loss function. The `loss_total.backward()` command will do a backward pass to see how the loss function changes with the model parameters (weights and biases), and apply the chain rule at every step in the calculation of `loss_total`. This results in a gradient which is *added* to the `.grad` attribute of every parameter. In other words, `loss_total.backward()` does not *save* the gradient, but *accumulates* the gradient. This means that the `.grad` attributes need to be set to zero before every backwards pass, which is exactly what `optimiser.zero_grad()` accomplishes. This statement is absolutely necessary, without it the gradients would be accumulated in the `.grad` attributes which causes the weights and biases to be up-

dated incorrectly.

`optimiser.step()` will read the `.grad` attributes and update the m_t and v_t values for every parameter and update the parameters accordingly (see eq. 2.14). m_t and v_t are not zeroed by the `optimiser.zero_grad()` command, because otherwise it would be impossible to implement the Adam optimiser since the gradients at previous epochs are needed.

`scheduler.step()` changes the learning rate according to the specific scheduler used. In this case, an internal counter of the number of epochs is kept, and once this counter reaches a multiple of 3000, the learning rate is halved.

The order of these commands are crucial. The gradients must be set to zero first to avoid accumulation of gradients. The gradients must then be calculated to avoid using the gradients of the previous epoch to update the parameters. Finally, the fresh gradients are used to update the parameters. [7]

Question 7

Why do the collocation time input tensor and the initial condition time input tensor require the gradient option, while the input data tensors do not require a gradient?

```
t_obs_t = to_tensor(t_obs)
y_obs_t = to_tensor(y_obs)
t_col_t = to_tensor(t_col, requires_grad=True)
t_ic_t = to_tensor(t_ic, requires_grad=True)
```

The collocation points are used to calculate the MSR of the ODE as shown in equation 2.4, and the initial conditions are used to calculate the deviation from the predicted initial conditions as shown in equation 2.5. In both expressions of the loss function, the derivatives of $y(t)$ show up, so these need to be calculated.

Rather than using finite differences to calculate the derivatives at the collocation points and at $t = 0$, we can calculate the gradients during the forward pass through the model using `torch.autograd.grad`. This is more efficient since it allows for the simultaneous prediction of $y(t)$ and the calculation of the derivatives. However, we need to tell the model which derivatives with respect to t are required, which is done by the `requires_grad=True` statement. For the calculation of the MSE of the noisy observation points as in equation 2.3, no derivatives are needed, so the statement is omitted and `requires_grad` is set to the standard value `False`. [7]

Question 8

When presenting arrays of time points to the input of my neural network, I need to convert them into a pytorch tensor, using this helper function:

```
def to_tensor(arr, requires_grad=False):
    return torch.tensor(arr, dtype=torch.float32,
                        requires_grad=requires_grad).unsqueeze(1)
```

What does the `unsqueeze(1)` method of the `torch.tensor` class do, and why do I need to do this?

The `unsqueeze()` method takes a pytorch tensor of rank n and adds a dimension of depth 1 at a specified index, as shown on Figure 2.2. In this case, the helper function takes in a one-dimensional array of length

n and returns a $(n, 1)$ `torch.tensor` class. This is necessary for two different reasons:

1. The time arrays (`t_obs_t` and `t_col_t`) which are used as inputs for forward passes through the network need to be $(n, 1)$ shaped in order to do matrix multiplication. The first `nn.Linear(1, 32)` layer of the network performs the following matrix multiplication:

$$y = xW^T + b \quad (2.15)$$

where x is the $n \times 1$ input of the first layer, consisting of n samples with 1 feature, which is the time value t . W and b are (32×1) column matrices containing the weights and biases¹ of the layer and y is the $n \times 32$ output. If we do not unsqueeze the time arrays, which correspond to x in this case, `nn.Linear(1, 32)` will interpret the $(n,)$ tensors as $(1, n)$ tensors, consisting of 1 sample with n features. This makes the matrix multiplication xW^T , which is $(1, n) \times (1, n)$ impossible. Note that the `t_ic_t` tensor does not need to be unsqueezed, since it is a $(1,)$ shaped tensor which cannot be interpreted incorrectly by the `nn.Linear(1, 32)` layer.

2. The `y_obs_t` unsqueezed tensor is not passed through the network, but rather is used to calculate the data loss $\mathcal{L}_{data}$ (see equation 2.3). When predicting the y values for the `t_obs_t` values, the model returns a $(n, 1)$ tensor (`y_pred`). In the code, the residual is calculated as `l_data = torch.mean((y_pred - y_obs_t) ** 2)`. If `y_obs_t` is not unsqueezed ($(n,)$ tensor), the $(n, 1) - (n,)$ operation will return a (n, n) tensor, containing *all* differences $\hat{y}_i - y_j$, instead of only the relevant errors $\hat{y}_i - y_i$. When averaging over these elements using `torch.mean`, it is averaging over differences and returns a loss which is not correct.

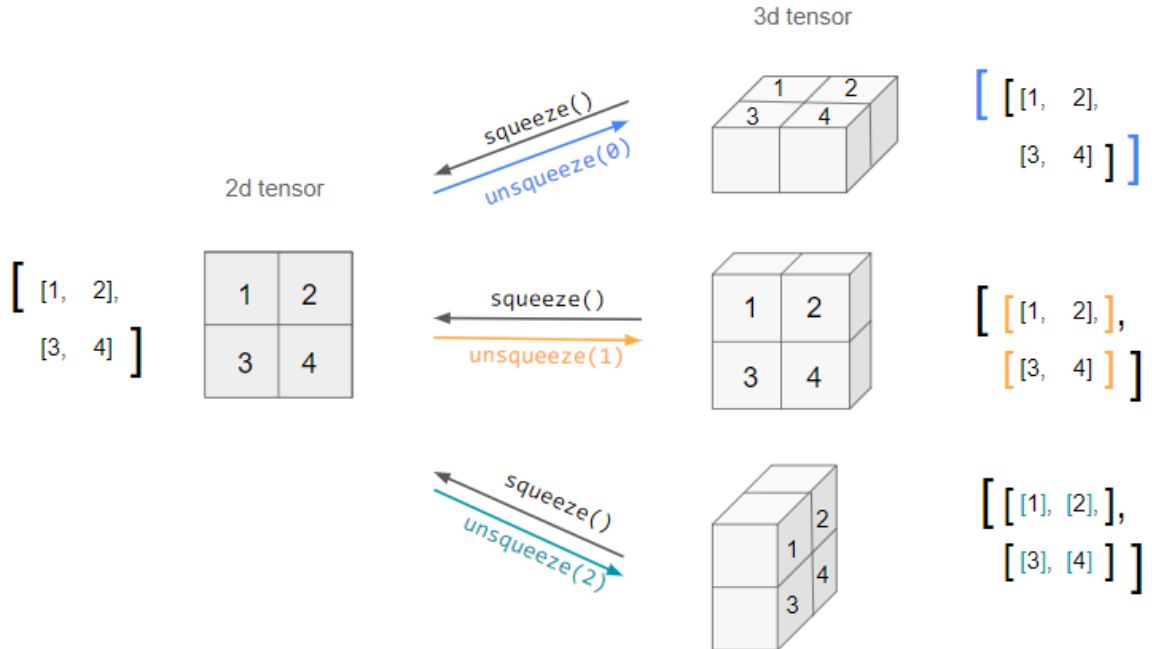


Figure 2.2.: Unsqueezing of a 2×2 tensor into $1 \times 2 \times 2$, $2 \times 1 \times 2$, and $2 \times 2 \times 1$ tensors by using the `unsqueeze()` method of the `torch.tensor` class. [8]

¹ b cannot actually be added directly to xW^T since the dimensions do not match. b is actually added to each row of the xW^T matrix.

Question 9

When computing the first and second derivatives of $y(t)$ we use the `torch.autograd` method. Explain why the `grad_outputs` are set to ones and why we need to take the zero-th element `[0]` of this output in the statements below:

```
dy = torch.autograd.grad(
    y_hat, t,
    grad_outputs=torch.ones_like(y_hat),
    create_graph=True)[0]
```

The `torch.autograd.grad` differentiates an output vector $y = (y_1, \dots, y_n)$ with respect to an input vector $t = (t_1, \dots, t_n)$ and returns a vector-Jacobian product:

$$v^T J = v^T \begin{pmatrix} \frac{\partial y_1}{\partial t_1} & \dots & \frac{\partial y_1}{\partial t_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_n}{\partial t_1} & \dots & \frac{\partial y_n}{\partial t_n} \end{pmatrix} \quad (2.16)$$

where v is set by `grad_outputs`. In our specific case, every value y_i only depends on the corresponding time value t_i , which makes the Jacobian a diagonal matrix. In order to contract the Jacobian to a column vector, v can be set to a row vector of ones with `grad_outputs=torch.ones_like(y_hat)`. The `torch.autograd.grad` actually returns a tuple $(dy,)$ per input, so to unpack it we can simply select the first and only element using `[0]`.

2.3. Improving the example code

2.3.1. Code improvements

2.3.2. Code demonstration

2.3.3. Hyperparameter optimisation

2.4. Inverse problem

2.4.1. Underdamped case

2.4.2. Critically damped case

2.4.3. Overdamped case

2.4.4. Statistical analysis

2.4.5. Discussion

3. Burger's Equation

3.1. Ground truth solving strategies

3.1.1. Numerical PDE solvers

3.1.2. Cole-Hopf Transformation

3.2. PINNs for the Burger's equation

3.2.1. Code implementation

3.2.2. Code demonstration

3.2.3. Hyperparameter optimisation

3.3. Inverse problem

3.3.1. Underdamped case

3.3.2. Critically damped case

3.3.3. Overdamped case

3.3.4. Statistical analysis

3.3.5. Discussion

4. Conclusion

A. Equation derivations

B. Figures

Bibliography

- [1] GeeksforGeeks. Loss function for linear regression in machine learning, 2024. Accessed: 2026-05-06. 4
- [2] GeeksforGeeks. Adam optimizer, 2025. Accessed: 2026-05-07. 6
- [3] GeeksforGeeks. Momentum-based gradient optimizer, 2025. Accessed: 2026-05-07. 6
- [4] GeeksforGeeks. Rmsprop optimizer in deep learning, 2025. Accessed: 2026-05-07. 6
- [5] GeeksforGeeks. Hyperparameter tuning, 2026. Accessed: 2026-05-07. 5
- [6] Y. Guo, Z. Fu, J. Min, S. Lin, X. Liu, Y. F. Rashed, and X. Zhuang. Long-term simulation of physical and mechanical behaviors using curriculum-transfer-learning based physics-informed neural networks. 2025. Accessed: 2026-05-07. 6
- [7] PyTorch Contributors. *PyTorch Documentation, Version 2.11*. PyTorch, 2025. Accessed: 2026-05-12. 10
- [8] Stack Overflow contributors. What does unsqueeze do in PyTorch?, 2019. Accessed: 2026-05-12. 11
- [9] G. van Rossum, J. Lehtosalo, and Ł. Langa. PEP 484 – type hints. Python Enhancement Proposal 484, Python Software Foundation, 2015. Status: Final. Python version: 3.5. 8